

ael Weeks	Director, Data and Analytics	
Functional Lead - CSM Success Management	Sherrod Patching	VP, Customer
Functional Lead - CS Strategy & Ops Director, CS Strategy and Operations	Brittney Sinq	Senior
Functional Lead - Product Product Management	Jackie Porter	Director,
Functional Lead - Product Product Management	Sam White	Group Manager,
Functional Lead - Product Product Management	Sarah Waldner	Group Manager,
Functional Lead - Product Director, Product Management	Justin Farris	Senior
Functional Lead - Product Product Management	Sam Kerr	Group Manager,
Member - Customer Success Operations Manager	Michael Arntz	Sales Strategy
Member - Customer Success Operations Customer Success Operations Analyst	Nishant Khanna	Senior
Member - Customer Success Operations Analytics Analyst	Brandon Butterfield	Senior Sales
Member - Product Management	Mike Flouton	VP of Product
Member - Product Director, Product Management	Hillary Benson	Senior
Member - Product Manager, Product Management	Tanuja Jayarama Raju	Product
Member - Product Manager, Product Data Insights	Carolyn Braza	Senior
Member - Enterprise Data Analytics Engineer	Michelle Cooper	Senior
Member - Enterprise Data Analytics Engineer	Miles Russell	Senior
Member - Enterprise Data Enterprise Data and Analytics	Amie Bright	VP of
Member - Enterprise Data	Jong Lee	Data Analyst
Member - Enterprise Data Manager, Data	Naheil McAvinue	Senior
Member - Office of the CEO Strategy and Operations	David Tuan	Director,

Jobs to be done by the Working Group

Job to be Done	DRI(s)	
Department		
----- ----- -----		

Opportunity canvas and defining outcomes Success	Brittney Sinq, Sherrod Patching	Customer

Opportunity canvas and defining outcomes Management - CI	Jackie Porter, Mike Flouton	Product
Opportunity canvas and defining outcomes Product Management - Compliance	Hillary Benson, Sam White, Sarah Waldner	
Opportunity canvas and defining outcomes Product Management - Security	Hillary Benson, Sarah Waldner, Sam White	
Instrument New Metrics	Jackie Porter, Sam White, Sam Kerr, Sarah Waldner	Product Management - CI, Security, Compliance, Analytics Instrumentation
Defining Lighthouse Metrics in the Hanbook	Israel Weeks	
Enterprise Data		
Build and publish certified Tableau dashboard	Naheil McAvinue	
Enterprise Data		
Build dbt data models	Israel Weeks	
Enterprise Data		

Multimodal Communication

- Apply the following labels to issues and epics to enable tracking and team wide visibility
 - WorkingGroup::LighthouseMetric
 - UseCase::CI
 - UseCase::Security
 - UseCase::Compliance
- Apply the appropriate Working Group status labels for issues or epics
 - wg-status::Not Started
 - wg-status::Ready
 - wg-status::In Progress
 - wg-status::Blocked
 - wg-status::Complete
 - wg-status::Cancelled
- Formalize "asks" of the Working Group by filing an issue against the EPIC to enable cross-functional visibility, collaboration, and prioritization

SECTION: company/working-groups/log-aggregation.md

Attributes

Property	Value
----- -----	
Date Created	March 6, 2019
Date Ended	July 2019
Slack	wglog-aggregation (only accessible from within the company)
Google Doc	Log Aggregation Working Group (only accessible from within the company)
Issue Label	WG-LogAgg (gitlab-com/-org)

Business Goal

Increase the quality, value, and accessibility of our GitLab.com logs.

Intent is to:

1. Analyze and document the locations, retention, and variety of production logs as they exist today, including analysis of the data classification and access controls
1. Develop troubleshooting and investigation guides to make best use of GitLab.com logs
1. Perform gap analysis of log quality and completeness, and where improvements can be made to streamline investigations
1. Work with Infra and Development teams to integrate the proposed improvements into GitLab.com

Exit Criteria

1. Completed audit of existing log sources, storage locations, and means of accessing the aggregated logs along with a quality and accuracy evaluation of these logs
1. Creation and delivery of a standardized logging lifecycle providing:
 1. Contribution: The how, when, and why Developers should add logged events to their code
 1. Production: Ensuring each service as part of the GitLab package is writing logs in a consistent and configurable fashion
 1. Aggregation: The consistent, reliable, and configurable collection and shipping of logs to defined location(s)
 1. Accessibility: Tooling and functionality to provide timely access and manipulation of the log data
 1. Retention: Define, provide means to enforce, and verify the necessary retention policies on log data
1. Publish documentation and training material on the GitLab logging lifecycle for use by the GitLab.com service and self-managed GitLab

Roles and Responsibilities

Working Group Role	Person	Title	
-----	-----	-----	
Facilitator	Paul Harrison	Senior Security Engineer	
Member	Stan Hu	Engineering Fellow	
Security Lead	Antony Saba	Senior Threat Intelligence Engineer	
Infrastructure Lead	Andrew Newdigate	Staff Engineer, Infrastructure	
Member	Amar Amarsanaa	Site Reliability Engineer	
Member	Alex Groleau	Security Software Engineer, Automation	
Member	Shawn Sichak	Security Engineer, Operations	

SECTION: company/working-groups/logging.md

Attributes

Property	Value	
-----	-----	
Date Created	2022-06-15	
Date Ended	TBD	
Slack	wg-glsecurity-logging	
Google Doc	Logging Working Group	

| Issue Label | WGLogging |

Business Goal

1. Increase efficiency by having a highly available simplified logging structure used by GitLab security to monitor, manage, detect, alert, respond, and audit events.
1. Increase collaboration by having a simplified logging system for GitLab security to share and collaborate on events.
1. Increase results by having relevant and actionable data for GitLab Security to make more accurate and informed decisions to threats to GitLab

Exit Criteria

1. Identify existing infrastructure, known gaps, and current security logging efforts as of August 2022.
1. Define a process for tracking existing, new, and retired security logging infrastructure.
1. Define logging profiles that include data requirements, storage, retention, and compliance / legal requirements.
1. Identify next steps, shared OKRs, other initiatives beyond the scope of the working group to carry the work forward.

Outcome

Other Investigations

Moved Here

Roles and Responsibilities

Working Group Role	Person	Title	
-----	-----	-----	
Facilitator	Harjeet Sharma	Security Engineer, SIRT	
Member	Jeff Martin	Senior IT Systems Engineer	
Member	Steve Manzuik	Senior Manager, Threat Management	
Member	Byron Boots	Senior Security Compliance Engineer	
Member	Kyle Smith	Senior Security Assurance Engineer	
Member	Paulo Martins	Infrastructure Security Engineer	
Member	Jayson Salazar	Staff Security Engineer, Security Automation	
Member	Dan Croft	Senior Engineering Manager, Ops	
Member			

Requirements and Considerations

Actors

- Security Operations
- Infrastructure Security
- Product Security
- IT Engineering
- Infrastructure

- Development
- Product
- Legal
- Quality

General

- As a GitLab team member, I know there is a single team I can talk to about all things being logged at GitLab.
- As a GitLab team member, I can find who is on the logging team by visiting a single handbook page.
- As a GitLab team member, I can contact the logging team via dedicated slack channel.
- As a GitLab team member, I can label GitLab issues for the logging team.
- As a logging team member, I only have one set of logging services/infrastructure to manage and maintain.

Security Assurance

- As a Security Assurance team member, I can easily find, search, view, and analyze logs up to 12 months old

SIRT Engineer

- As a SIRT Engineer, I can easily find, search, view and analyze logs for up to 1 year
- As a SIRT Engineer, I can retrieve archived log sources that are over 1 year old and be able to easily search, view and analyze those logs
- As a SIRT Engineer, I can easily create rules, alerts and dashboards for Incident Response.
- As a SIRT Engineer, I can find in the handbook information about log sources including:
 - The DRI's responsible for each log source
 - The format of each log source
 - The fields recorded in each log source
- As a SIRT Engineer, I am able to easily integrate logging data into other SIRT tools for fast, accurate and reliable Incident Response

Infrastructure Security

- Collection
 - As an Infrastructure Security Engineer, I need it to be easy for anyone to setup and ship audit logs from their infrastructure or third party tools to a centralized location
 - As an Infrastructure Security Engineer, I need to be able to easily export findings from IDS tools
- Delivery
 - As an Infrastructure Security Engineer, I need our logs to be scalable, consistent and reliable (no broken timestamps, no loss of data)
- Retention
 - As an Infrastructure Security Engineer, I need our logs to comply with Retention & Disposal Requirements, compliance with internal policies, standards, and regulatory requirements
- Monitoring and Alerting
 - As an Infrastructure Security Engineer, I need to be able to easily analyze logs and threat findings for one or more infrastructure resources

- As an Infrastructure Security Engineer, I need to be able to create alerts on rules and anomalies from my logs
- Change Management
 - As an Infrastructure Security Engineer, I need all the infrastructure and configuration to be managed as code (terraform, python, bash), and change management following the Merge-Request-Review process

IT Engineering and Infrastructure

- As an IT Engineer, I can easily add a tech stack application or vendor SaaS service to the logging service and start shipping logs to, as easily as we do for bug reporting with Sentry today.
- As an IT Engineer, I can configure basic downtime or high activity threshold alerts that send to different notification services (email, Slack, GitLab issue, etc.)
- (Optional) As an IT Engineer, I can see end-to-end system logs to investigate all affected or related systems during an incident by simply changing the data source from a drop down menu, while retaining or easily re-creating the same search criteria (actor, IP, date range, etc).
- As an IT Engineer, I can create a hierarchical organization of services that are logged to support related technologies that allows us to create alerts and dashboards for a group of interconnected services.
- As an IT Engineer, I want to support for multiple instances of the same product/service without having to re-create each instance and all policies by hand.
- As an IT Engineer, I am concerned about integration of new technologies and reliance on vendors to add support for integrations several months or years later, or requiring Professional Services. I would prefer to an open API ecosystem approach that encourages our products/service providers to build integrations with our logging and monitoring provider.
- As an IT Engineer, I want to be able to use API calls, configuration management, and/or IaC (Terraform/Ansible) to standardize configurations and provisioning.
- As an IT Engineer, I do not want to have to create a dashboard or alert for every request that a team member has. The platform should be easy to use from a building query and report perspective, comparable to building a pivot table in Excel or a report in Tableau/SiSense with easy-to-identify fields. Complex SQL query language is useful behind the scenes, but discourages adoption of proactive alerting and is only useful for specific queries during incident triage.
- As an IT Engineer, I want to see this program evolve into a organization-wide NOC dashboard for our broader Tech Stack applications and services that each team supports.

SECTION: company/working-groups/maintainership.md

Attributes

Property	Value
Date Created	2022-04-19
End Date	2023-02-02
Slack	wgmaintainership
Google Doc	Working Group Agenda
Task Board	Issue board

Business Goal

We have seen overall inconsistent results with maintainership in the last quarter. Examples: A subset of maintainers are taking the burden of reviews which can lead to serious problems in job satisfaction issues and burnout. We are growing (both in headcount as well as community contributions), but the number of maintainers has stabilized. The number of repos which need maintainer support is increasing while coverage of them has decreased. We want transparency that seniors who are maintainers are having a positive impact in the multiple areas listed here, which leads to more career opportunities for them than it does non-maintainers.

Our objective is to change our processes and culture to have an organization which we know can sustain maintainership for the next 5 years that meets the demand of both the company and the open core project. This includes, but is not limited to:

- Increasing current maintainers and having a forecasting to know we will increase in the future.
- Availability measures which demonstrate maintainers are able to meet demands of code reviews
- Load balancing measures to distribute MR reviews evenly among maintainers
- Improvements in code review features and CI/tooling to support the maintainers and reduce work needed for reviews
- Coverage/monitoring measures to know when a part of the code base is adequately supported or needs help
- Improvements in onboarding maintainers at our new scale
- Have some fun

Tracking Progress

Progress will be tracked on the Working Group issue board using the following labels:

- ~"workflow::In dev"
 - The issue is currently in progress and actively being worked on
- ~"workflow::In review"
 - The issue is currently being reviewed by broader Engineering Team
- ~"workflow::blocked"
 - The issue is blocked by another issue. Please refer to the blocking issue.
- ~"workflow::production"
 - The issue has been completed and should be closed.

Exit Criteria (100% completed)

Exit criteria for this working group has been completed, however monthly follow up meetings are still active until February 2, 2023.

Maintainership Process

The maintainership process is described on this page.

Exit Criteria (100% completed)

	Start Date	Target Completion Date	Completed Date	DRI	Criteria
	--	-----	-----	-----	-----
1	2022-06-01	2022-07-22	2022-10-28	@nhxnguyen	Create an implementation plan to remedy gaps in Maintainership coverage
2	2022-04-26	2022-07-22	2022-07-18	@mwoolf	Develop metrics to provide more transparency into the health of the Maintainership program
3	2022-05-04	2022-08-05	2022-10-04	@robotmaygitlab	Update expected behaviors and responsibilities for Engineers and Maintainers
4	2022-05-18	2022-08-05	2022-10-04	@oregand	Improve the Trainee Maintainer process to make the process more efficient
5	2022-06-01	2022-08-05	2022-11-16	@sabrams	Develop and implement a communication plan for Maintainership changes

Data and dashboards

- Reviewer/Maintainer availability and capacity - Shows the maintainer/reviewer availability rate over time and incoming/forecasted review requests
- Maintainers and Trainees
- Maintainership Demand

Roles and Responsibilities

Working Group Role	Person	Title
Executive Sponsor	Christopher Lefelhocz	VP of Development
Facilitator	Michelle Gill	Senior Engineering Manager, Manage
Functional Lead (Enablement)	Alex Ives	Engineering Manager, Database
Functional Lead (Fulfillment)	Jerome Ng	Senior Manager of Fulfillment
Functional Lead (Ops)	Sam Goldstein	Director of Ops
Functional Lead (Dev)	Max Woolf	Senior Backend Engineer, Govern:Compliance
Functional Lead (Sec, Data Science, Growth)	Thomas Woodham	Sr. Engineering Manager, Secure Analyzers
Functional Lead (Maintainer - Frontend)	Natalia Tepluhina	Staff Frontend Engineer
Functional Lead (Non-Maintainer - Backend)	Manoj M J	Senior Backend Engineer
Functional Lead (Trainee - Registry DB)	Steve Abrams	Intermediate Backend Engineer
Functional Lead (Maintainer - Workhorse, Shell)	Robert May	Senior Backend Engineer
Functional Lead (Maintainer - Frontend)	Ezekiel Kigbo	Senior Frontend Engineer
Functional Lead (Maintainer - Omnibus)	Balasankar C	Senior Backend Engineer
Functional Lead (Maintainer - CNG, Operator)	Mitchell Nielsen	Senior Backend Engineer

Member	Sean McGivern	Staff Backend	
Member	Allen Cook	Senior Backend	
Member	Terri Chu	Senior Backend	
Member	Doug Stull	Staff Fullstack	
Member	Pavel Shutsin	Senior Backend	
Member	Sincheol Kim	Senior Backend	
Member	Micha· Zaj·c	Senior Backend	
Member	Douglas Barbosa Alexandre	Staff Backend	
Member	Paul Gascou-Vaillancourt	Senior Frontend,	
Member	Dennis Tang	Engineering Manager, Govern:Compliance	
Member	Nick Nguyen	Senior Engineering Manager, Datastores	
Member	Darva Satcher	Senior Engineering Manager, Create /	
Ecosystem Stage			
Member	Jiaan Louw	Senior Frontend Engineer, Govern:Compliance	
Member	Rémy Coutable	Staff Backend Engineer, Contributor Success	

Communicating changes

Labels

We have two sets of labels to help facilitate communications:

Type of changes and their impact

Use these labels to identify the type of change being made:

- ~"Maintainership WG::process change" - This update changes an existing process or workflow
- ~"Maintainership WG::new process" - This update introduces a new process or workflow
- ~"Maintainership WG::responsibility change" - This update changes or introduces a responsibility
- ~"Maintainership WG::other change" - This is an update that may warrant an announcement but does not fit in the above categories

Communication status

Use these labels to identify if a change is ready to be communicated or if it has been communicated:

- ~"Maintainership WG Comms::ToDo" - This update is ready to be communicated
- ~"Maintainership WG Comms::Needs Support" - This update needs something additional to support it when it is announced. This could be things like handbook updates, an FAQ, or an AMA.
- ~"Maintainership WG Comms::Done" - This update has been communicated

Runbook

Levels of announcements

There are generally three types of announcements that could apply to any update:

1. Alert - Something certain individuals might want to be aware of.

Example: letting trainees and maintainers know that the program is meant to last one year, at which time it should be evaluated.

Alerts are the lowest priority type of announcement. If some people miss it, that is not a problem. Alerts can usually be announced one time.

1. Change - Something certain individuals need to be aware of.

Example: changing the number of approvals necessary to become a maintainer.

Changes usually require adoption, which is not something that happens after a single announcement. So it is important to not only communicate to the effected individuals, but also those who can endorse and promote the changes such as engineering managers or higher levels of management.

Some announcements may gain a wider audience if they come from someone in senior management. If the subject matter is high enough in priority, consider reaching out to your sub-department director for guidance or asking in an engineering staff meeting for help with the announcement.

1. Action - Something that requires certain individuals to take action within a given time period of the announcement.

Example: requesting survey feedback or communicating a behavior change that needs to be adopted within a certain time frame.

Depending on the priority and urgency of the action needed, additional reminders are appropriate. For surveys, reposting a few times within the time frame before the survey closes is appropriate.

Making an announcement

Consider the following questions when making an announcement:

- Are there any other areas of the docs or handbook that should be updated? Ensure there is no conflicting information that could cause confusion.
- Do you have links to the handbook or docs to accompany the announcement? Ensure more information is available for those who need it.
- Do you have a place where people can go with more questions or clarifications? Provide an issue/doc/slack channel/AMA if it is expected that people may want further details.
- Does this change disrupt an established process? If so, it may make sense to prompt managers to take action to raise awareness and adoption of the process.
- Is the change large enough that we may have a large number of questions? Consider opening an FAQ file or scheduling an AMA (@sabrams can also facilitate these items).

- [] Include links to the most relevant piece of information. Do not overcrowd the announcement with non-primary links.
- [] If the announcement is for awareness:
 -] Communicate it to the most [relevant slack channel and cross-post to other relevant

channels

- ☐ Add it to eng-week-in-review
- ☐ If the announcement is for action
 - ☐ Communicate as above and consider adding it to management staff meetings.
- ☐ If you are seeking feedback as a result, provide a link to where feedback can be given

Divisive changes

Sometimes changes are controversial or involve subjects where people might have deep opinions. It is important that when such changes occur, everyone has access to resources and information to help them understand:

- How they are affected
- Why the change was made
- Where they can give feedback

An FAQ or AMA can be helpful in supplying much of this information.

Ideally, we want people to embrace a change, but at the least, we need to start with adoption. To help guide embracement, the "why" behind the change has to be strongly defined, optimally with strong data backing it.

Conducting a survey

- ☐ Establish the goal of the survey
- ☐ Share in the most [relevant slack channel and cross-post to other relevant channels
- ☐ Communicate the end date or due date for the survey
- ☐ Define a minimum number of responses to make the results representative/useful
- ☐ Add a question to distinguish between different subgroups if applicable (frontend/backend, reviewer/trainee/maintainer) for pivoting the results.

Communication channels

It is appropriate for almost any changes to be communicated in:

- development
- eng-week-in-review

Communicating in whats-happening-at-gitlab is also a good idea for changes that effect large groups of people.

Here are a few different categorizations of changes that can help you decide which channels to consider communicating to.

By exit criteria

| Exit Criteria | Relevant Channels |

| ----- | ----- |

| Create an implementation plan to remedy gaps in Maintainership coverage | Channels for the projects directly effected by the changes. |

Develop metrics to provide more transparency into the health of the Maintainership program	engmanagers Engineering staff meetings
Update expected behaviors and responsibilities for Engineers and Maintainers	traineemaintainers backendmaintainers frontendmaintainers databasemaintainers engmanagers Engineering staff meetings
Improve the Trainee Maintainer process to make the process more efficient	traineemaintainers backendmaintainers frontendmaintainers databasemaintainers engmanagers Engineering staff meetings

For gitlab-org/gitlab (GitLab Rails)

Topic	Relevant Channels
Backend	backend backendmaintainers
Frontend	frontend frontendmaintainers
Database	database databasemaintainers
Trainee updates	backendmaintainers frontendmaintainers traineemaintainers
Responsibility/process updates	Any of the above channels and engmanagers

Other projects

For changes concerning a specific project, try to notify the group that owns the project in addition to specific channels for that project. For example, workhorse changes should be communicated to workhorse and gcreatesourcecode as the owners of that project.

The product category page and the projects page may be helpful for determining which groups and stages own specific projects. If it is not apparent, another way to determine ownership include filtering the project members by direct membership and asking the owners or maintainers directly who owns the project.

SECTION: company/working-groups/major-releases.md

Attributes

Property	Value
-----	-----
Date Created	2024-02-22
End Date	2025-07-01
Slack	wgmajor-releases (only accessible from within the company)
Google Doc	Major Releases Working Group Agenda
Epic	Future Major Release Coordination & Improvements
Overview & Status	Closed

Closing update

This group is closed with the outcome of the group consolidating in and opinion on breaking changes that the org can adopt:

- Migrations over breaking things 2024 Executive Brief (internal)
- Migrations over breaking things 2024 SSlides (internal)
- Migrations over breaking things 2024 Recording (internal)

Purpose

The `Major Releases Working Group aims to minimise any negative customer impact related to the major release and breaking change across SaaS and self managed GitLab users.

Overview

The Major release working group will form from the point that all deprecations have been announced that are due for the major release according to the deprecation policy until we are at least one milestone past the major release and aware of any customer issues that have arisen as part of the major upgrade and have a plan of action. The rough schedule for this is outlined in the attributes table above.

Goals

This is a non-comprehensive list of topics to be discussed.

- Minimise customer impact due to breaking change and major release.
- Go through product deprecations and separate the high impact ones for further analysis.
- Prepare Support and CS for customer questions on deprecations & upgrade - Internal Slides.
- Release communications to prepare customers.
- Understand whether deprecation policy and GitLab's working interpretation are aligned / need to be changed.
- Put a plan in place to improve conditions for next major release and remove need for this working group next iteration.

Exit Criteria

Exit Criteria are available in the Future Major Release Coordination & Improvements Epic.

Roles and Responsibilities

Working Group Role	Person	Title
----- ----- -----		

Executive Sponsor	Sabrina Farmer (@sabrinafarmer)	CTO
Executive Stakeholder	Josh Lemos (@joshlemos)	CISO
Facilitator/DRI	Sam Wiskow (@swiskow)	Sr. PM, Delivery &
Scalability - Infra		
Product Management	Mike Flouton (@mflouton)	VP - Product
Member	Marin Jankovski (@marin)	Senior Director -
Infrastructure, SaaS Platforms		
Member	Fabian Zimmer (@fzimmer)	Product Director -
SaaS Platforms		
Member	Michele Bursi (@mbursi)	Engineering

Manager, Delivery Group |
 | Member | Steve Evangelista (@steve-evangelista) | Senior Product
 Director - Dev & Analytics |
 | Member | Martin Bruemmer (@mbruemmer) | Staff Customer
 Success Manager |
 | Member | Lyle Kozloff (@lyle) | Director of
 Support, Global Readiness |
 | Member | Stephen Denham (@denhams) | Engineering
 Manager, Environment Automation, Dedicated |
 | Member | Cheryl Li (@cheryl.li) | Senior Engineering
 Manager, Verify |
 | Member | Conley Rogers (@conleyr) | Staff Customer
 Success Architect |

SECTION: company/working-groups/merge-request-report-widgets.md

Attributes

Property	Value
Date Created	August 1, 2021
Target End Date	January 31, 2022
Actual End Date	August 18, 2022
Slack	wgmerge-request-report-widgets (only accessible from within the company)
Google Doc	Merge Request Report Widgets Working Group Agenda (only accessible from within the company)
Implementation discussion	gitlab333975
Pajamas documentation	Region: Merge request reports
Design specs	Pajamas UI Kit
Associated OKRs	product2872
DRIs for each Widget	DRI list

Charter

The working group ensures consistent alignment and development of all Merge Request (MR) Report Widgets. By synchronizing a collection of independently created widgets into a single report region, a code reviewer should have an easier time reviewing the merge request analysis and determining next steps.

The working group is tasked with creating a joint codebase where consistency is at the core of each widget. It's enforced and achieved by the codebase and established APIs.

Business goal

Drive forward the adoption goals through usability by facilitating improvements to merge requests that will positively impact the shared FY22-Q3 KR between Product and UX. By following the UX Framework, we ensure the level of maturity expected by customers dealing with merge requests. From now on, we will have clear and strict guidance to ensure sustainable and

consistent growth for the MR Widget ecosystem.

The main target of the working group is not to implement everything themselves but rather keep the effort moving across the frontend engineering and UX groups in a constant pace and pick up key tasks.

Scope and definitions

Merge Request Reports Region

- Area of the merge request with summaries of analysis done and proposed changes. This area resides in the Overview- tab, between the description and comments or system notes.</dd>

Merge Request Widget Extensions

- Each piece of software that presents summarized information for analysis performed.

Merge Request Widget Extension Component (docs)

- Reusable component to allow each extension to customize the presentation of the information while still ensuring a consistent user and developer experience for all extensions attached to the merge request.

Exit criteria

- Shepherd the implementation and redesign of 10 extensions (full list) that will be ported to fully use the new shared MR Widget Extension Component.
- The shared component follows the Report regions design guidelines defined in the Pajamas Design System.
- The documentation reflects the capabilities of the shared component.
- Ensure there is clear documentation written for extending the component.

Results

Documentation outcomes

- DRI List (Eng. and UX)
- Component technical documentation
- Contributing further and extending

Product outcomes

The screenshots below illustrate the Merge Request Report Widgets before and after the work delivered by the Working Group.

```
{{< cardpane >}}
{{% card header="Before" %}}
```

```
{{% /card %}}
```

{{% card header="After" %}}

{{% /card %}}

{{< /cardpane >}}

Roles and responsibilities

The functional leads will be responsible for:

- Representing the needs of individual stakeholders in their department/sub-dept.
- Gathering and consolidating feedback on specific proposals from their department/sub-dept.
- Communicating the output from the working group (if any) and answering questions from their dept/sub-dept.

Ideally, the functional lead is someone who is an IC working in the affected groups, but anyone capable of representing a group, department, or sub-department in the fashion mentioned above is welcome.

Working Group Role	Person	Stakeholder Dept.	Title
-----	-----	-----	-----
Executive Sponsor	Tim Zallmann	Dev	Director of Engineering, Dev
Facilitator	André Luís	Dev, Create:Code Review	Frontend Engineering Manager
Functional Lead	Tim Noah	UX	Senior Product Designer
Functional Lead	Phil Hughes	Create:Code Review	Staff Frontend Engineer
Functional Lead	José Iván Vargas López	Verify:Pipeline Execution	Senior Frontend Engineer
Functional Lead	Scott Hampton	Verify:Pipeline Security	Engineering Manager
Functional Lead	Savas Vedova	Govern:Threat Insights	Senior Frontend Engineer
Functional Lead	Mark Florian	Foundations	Senior Frontend Engineer
Functional Lead	Jannik Lehmann	Secure	Frontend Engineer
Functional Lead	Jeremy Elder	UX	Staff Product Designer
Member	Marcel van Remmerden	UX	Product Design Manager, Create
Member	Kai Armstrong	Product	Sr. Product Manager, Create:Code Review
Member	Rayana Verissimo	UX	Product Design Manager, CI/CD
Member	Payton Burdette	Verify:Pipeline Execution	Senior Frontend Engineer

Member	Pedro Moreira da Silva	UX	Staff Product
Designer, Create:Code Review			
Member	Tomislav Niki	Quality	Software Engineer
in Test, Create:Code Review			
Member	Gina Doyle	UX	Senior Product
Designer, Runner			

SECTION: company/working-groups/mit-mentoring.md

Attributes

Property	Value	
----- -----		
Date Created	May 11, 2020	
Target End Date	September 1, 2020	
Slack	wgmitmentoring (only accessible from within the company)	
Google Doc	Minorities in Tech (MIT) Mentoring Working Group Agenda (only accessible from within the company)	
Issue Board	Minorities in Tech Issue Board	

Business Goal

Deliver Minorities in Tech (MIT) Mentoring Program aligning with the MIT TMRG Mentorship Initiative.

Roles and Responsibilities

Working Group Role	Person	Title
----- ----- -----		

Facilitator	Sharif Bennett	Mid Market Account Executive
Engineering Lead	Darva Satcher	Create Engineering Manager, Editor & Knowledge
Digital Production Lead	Aricka Flowers	Manager, Digital Production
Executive Stakeholder	Christopher Lefelhocz	VP of Development
Executive Stakeholder	Ryan O'Neil	VP of Commercial Sales

Exit Criteria

The Minorities in Tech Mentoring Program Working Group will deliver:

- [x] Develop a Framework for an MIT Mentoring Program
- [x] Identify Participates
- [x] Provide Training for Participants
- [x] Run the first MIT Mentoring Program
- [x] Successfully graduate the first class of Mentor / Mentee Pairs

SECTION: company/working-groups/mlops.md

Attributes

Property	Value
Date Created	2021-08-04
Target End Date	2022-02-24
Slack	wgmlops (only accessible from within the company)
Google Doc	MLOPs Working Group Agenda (only accessible from within the company)
Board	gitlab-org board
Issue Label	~WorkingGroup::MLOps

Goals

This Working Group has the following goals:

1. Begin defining and refine the results driven business value stream of MLOps at GitLab.
1. Formalize the processes related to provenance, storage and access of GitLab.com production data for the purpose of model training.
1. Share knowledge and determine best practices for hyper-parameter tuning, retraining, versioning, and deploying new ML models
1. Determine best practices for benchmarking models built by different frameworks/libraries for different use cases in terms of accuracy and performance, and how to do it in a continuous basis.
1. Determine how to distribute machine learning models on self-managed instances
1. Define a security/legal process for security-related ML models and data pre-processing

Definitions

What is MLOps?

As per Wikipedia, MLOps or ML Ops is a set of practices that aims to deploy and maintain machine learning models in production reliably and efficiently.

Read more about the topic area from the links below:

- MLOps Wikipedia
- MLOps: Continuous delivery and automation pipelines in machine learning
- Machine Learning Operations
- MLOps Slack Group

Related GitLab Documentation

- MLOps Single Engineer Group
- MLOps Primer

- MLOps Exploration
- Product Stage Direction - ModelOps
- Draft Group Direction - MLOps
- Group Direction - AI Model Validation
- AI Model Validation Group

Related GitLab projects

- Spamcheck
 - Architecture Diagram
- Tanuki-Stan
- UnReview
 - Architecture Diagram

Related ML Slack channels

- gappliedml
- gmachine-learning
- feedtanuki-stan
- security-ml

Exit Criteria

The charter of this working group is to bridge the gap between different teams that are building ML products at GitLab by discussing overlapping architectural concerns:

1. Creation of a mission statement that the MLOps Working Group operates under, added to the handbook.
1. Create a handbook page discussing a deep analysis of the tooling for ML out there and a proof-of-concept framework using an ideal pathway that GitLab teams can refer to and easily extend when kicking off ML-related projects.
1. Creation of helpers/libraries (presumably in Python) that can be shared across teams and used for the purposes of similar tasks e.g. data access and storage, data pre-processing.

Outcome

1. The Working Group has brought together various teams that are working on different ways to leverage Machine Learning within GitLab the product and in their day-to-day work. It has given these teams a platform to discuss their mission and an opportunity to share knowledge on projects they are currently undertaking and what they are looking to achieve in the near future.
1. We will move forward with the creation of a monthly MLOps meetup that focuses on sharing ideas and showcasing work done by different teams, in a less formalized structure, to enhance awareness across the company.

Roles and Responsibilities

Working Group Role	Person	Title
-----	-----	-----

---|

Facilitator	Alex Groleau	Security Automation Manager
Functional Lead	Roger Ostrander	Senior Security Engineer, Trust & Safety
Functional Lead	Alexander Chueshev	Senior Backend Engineer, AI Framework
Functional Lead	Taylor McCaslin	Group Manager, Product - Data Science
Functional Lead	David DeSanto	Senior Director, Product Management - Dev & Sec
Functional Lead	Ethan Urie	Senior Backend Engineer, Security Automation
Functional Lead	Jayson Salazar	Senior Security Engineer, Security Automation
Functional Lead	Juliet Wanjohi	Security Engineer, Security Automation
Functional Lead	Eduardo Bonet	Staff Full Stack Engineer - MLOps, SEG
Functional Lead	Monmayuri Ray	Engineering Manager, AI Model Validation
Functional Lead	Shawn Sichak	Senior Security Engineer, Trust and Safety
Member	Alexander Dietrich	Senior Security Engineer, Security Automation
Member	Charl De Wit	Security Manager, Trust & Safety
Member	Wayne Haber	Engineering director
Member	Bartek Marnane	VP, Incubation Engineering
Member	Marshall Cottrell	Strategy and Operations (Technical) Staff
Member	Kelly Chen	Analytics Manager at Finance
Member	Jay Stemmer	Manager, Analytics & Insights

SECTION: company/working-groups/modern-applications-gtm.md

Attributes

Property	Value	
----- -----		
Date Created	June 20, 2022	
Target End Date	Dec 30, 2022	
Slack	wgmodern-applications-gtm (only accessible from within the company)	
Google Doc	Working Group Agenda (only accessible from within the company)	
Docs	Modern Applications Message House	
Related Issue		
Associated OKRs		

Problem To Solve

- While GitLab provides the necessary capabilities to deliver modern applications effectively,

the field lacks the artifacts and execution to deliver appropriate modern application messaging and guidance to our prospects and customers.

- Typically, we cannot identify application initiatives associated with our prospect's / customer's interest in GitLab, which limits our reach into the account and our ability to tie our value to a c-level initiative.
- GitLab ranks relatively low when searching for "microservices" and "modern applications," which could signify a lack of thought leadership in a domain we have immense value to provide.

Business Goal

- Establish GitLab as a thought leader and go-to partner for modern applications to generate more leads.
 - Enable the field to confidently and credibly position GitLab for modern application use cases leading to first-order wins.
 - Attach to strategic initiatives in prospect/customer accounts to drive larger deal sizes.
- Generate compelling customer stories by driving value for our customer's digital transformation and modern application initiatives.

Exit Criteria

- Message House Completed and confirmed by Product Marketing
- Pitch Deck Created

Step 1

Roles and Responsibilities

Working Group Role	Person	Title
Executive Sponsor	Nima Baidley	Vice President, Alliances
Facilitator	Jonathan Fullam	Senior Director, Solutions Architecture
Functional Lead	Kris Nelson (Ent sales)	Area Sales Manager
Functional Lead	Aathira Nair (Product Marketing)	Product Marketing Manager
Functional Lead	Lee Faus (Strategic Field Team)	Field CTO
Member	Reshmi Krishna	Sr Manager, Solutions Architecture

Meetings

This working group meets bi-weekly (every other week.)

Meetings are recorded and available in the Google document for the meetings.

SECTION: company/working-groups/multi-large.md

Attributes

Property	Value
Date Created	June 22, 2020
End Date	December 12, 2022
Slack	wgk8s-migration (only accessible from within the company)
Google Doc	Working Group Agenda (only accessible from within the company)
Overview & Status	GitLab.com on Kubernetes Epic

Charter

The charter of this working group is to synchronize cross department efforts, as a means to support GitLab.com migration to the Kubernetes platform. This migration is essential as it is a base for other initiatives which require running multiple, large independent GitLab sites without significant staffing demand.

Scope and Definitions

In this context, the scope of this working group is defined by the following list outlines the order of deliverables per Working Group Agenda on 2020-07-13, and then further refined on 2021-02-01. The goal of this list is to define deliverables for the Working Group such that its charter can be finite.

Deprecate NFS

1. Make Web/API work without NFS (issue tracker)
1. Make CI/CD live traces work without NFS (issue tracker)
1. Make Pages work without NFS (Scalability: <https://gitlab.com/groups/gitlab-org/-/epics/3980>)

Helm Charts

1. Move Web/API nodes of .com to Helm charts (issue tracker)
1. Move Pages nodes of .com to Helm Charts (issue tracker)

Stateful Nodes

1. Gitaly to K8s
1. Redis to K8s

Note that not all stateful nodes are scoped here, notably PostgreSQL, and this is intentional.

Roles and Responsibilities

Working Group Role	Person	Title
Executive Stakeholder	Steve Loyd	VP of Infrastructure
Facilitators	Chun Du, Michele Bursi	Director of Engineering, Enablement and Engineering Manager, Delivery
Functional Lead	Nailia Iskhakova	Software

Engineer in Test, Database			
Functional Lead		Andrew Thomas	Principal
Product Manager, Enablement			
Functional Lead		Gerardo "Gerir" Lopez-Fernandez	Engineering
Fellow, Infrastructure			
Member		Jason Plum	Staff Engineer,
Distribution			
Member		Tanya Pazitny	Quality
Engineering Manager, Enablement			
Member		Mek Stittri	Director of
Quality Engineering			
Member		Marin Jankovski	Sr Engineering Manager, Infrastructure, Delivery & Scalability
Member		Dilan Orrino	Senior Product Manager, Distribution
Member		Mark Wood	Senior Product Manager, Create:Gitaly
Member		Andras Horvath	Engineering Manager, Gitaly

Outcome

The Kubernetes Migration Working Group was established to reduce operational costs and increase the efficiency of day-to-day operations.

The working group defined the migration path of each service together with the responsible teams. This collaboration included analysis of a target workload in order to ensure the successful migration of the service into a Cloud Native architecture. Part of the effort was also the work on all related services of the supporting infrastructure needed to make migrations successful, like new Observability solutions and new release and deployment capabilities.

The migration took care of all the stateless services and some selected stateful ones. It was deliberately decided to not attempt the migration of Gitaly in light of a new architecture that will take place in future to avoid spending effort in migrating components that will be soon replaced. The work done in identifying a possible migration path still stands and it will be re-evaluated when the new architecture will be partially in place. The goal to revisit this is set for the end of FY24Q2.

The migration of services to Kubernetes allowed GitLab to optimize the infrastructure size and cost while providing a more flexible and efficient way of scaling services. In the same way, we have been able to maintain comparable or, in some cases, better performance. In addition to better performance, another success factor is deployment speed, where we observed approximately 1.5x improvement.

Migrating GitLab.com to run on Kubernetes also meant creating new installation methods available to our customers. We are dogfooding and continuously improving these new installation methods so also self-managed customers can benefit from it.

Timeline of Migrated Services

Date	Service	
-----	-----	
2019-08-30	Container Registry	

2019-09-27	PlantUML			
2019-11-21	Mailroom			
2020-05-30	Sidekiq Memory-bound Shard			
2020-05-15	Sidekiq Elasticsearch Shard			
2020-06-09	Sidekiq Low-urgency-cpu-bound Shard			
2020-06-16	Sidekiq Urgent-other Shard			
2020-07-07	Sidekiq Database-throttled Shard			
2020-07-07	Sidekiq Gitally-throttled Shard			
2020-07-14	Sidekiq Urgent-cpu-bound Shard			
2020-08-15	Websockets			
2020-10-26	Git HTTPS			
2020-12-01	Git SSH			
2021-06-28	API Service			
2021-07-13	Remaining Sidekiq Queues			
2021-07-22	Enhanced Services Observability			
2021-09-23	Web traffic on Kubernetes			
2021-12-01	Pages			
2022-08-05	Camoproxy			

The full list of the work accomplished to support the migration can be found on the [GitLab.com](#) on Kubernetes tracking epic.

Related Links

- Working group kick off meeting
- Defining Day-2 Operations - DZone Agile
- Scalability Team - GitLab

SECTION: company/working-groups/next-architecture-workflow.md

Attributes

Property	Value	
-----	-----	
Date Created	July 1, 2022	
End Date	December 31, 2022	
Slack	architecture (internal)	
Google Doc	Working Group Agenda (internal)	

Problem Statements

1. Cross-functional prioritization framework allocates some amount of time towards maintenance activities. It is not prescriptive regarding how Engineering Managers and Engineers might coordinate across multiple teams to execute on large-scale cross-functional projects like those defined by architecture evolution blueprints.
1. Architecture Evolution Workflow is complex, and has many steps that make it difficult to adopt the workflow and to succeed with

with using it to deliver larger, long-term initiatives.

1. In its current form, the Architecture Evolution Workflow is not leveraged for many initiatives within departments of Engineering. Without a unified roadmap of our own, it is difficult to weigh priorities against Product's own roadmap.

Exit Criteria

The charter of this working group is to define and implement the next iteration of the Architecture Evolution Workflow.

We want to make it easier and more intuitive to use the Workflow, but also to improve the process itself, to position team members better for delivering great results by using it.

- Firm up the definition of "Architecture" and "Blueprint" in this context.
- Take inventory of current and historical Engineering Allocation initiatives and evaluate the extent to which such initiatives could be defined as blueprints, following the Architecture Evolution Workflow.
- Lower the entry barrier for creating a proposal in a blueprint.
- Establish criteria for when a blueprint should be used.
- Identify cross-functional touchpoints and consolidate upstream processes like production readiness, AppSec reviews, and creation of release posts.
- Develop strategy for incorporating this process and the Engineering roadmap into Product planning and prioritization via the Cross-functional Prioritization framework.

Context

- The Architecture Evolution Workflow is complex, spans multiple tools and projects, and is not something external contributors can participate in.
- Architecture Blueprints provide great context and detail, but they are completely disconnected from Product prioritization and thus rarely picked up into the workstream.
- Working Groups involving technical initiatives have a low rate of success. Many choose to produce blueprints as part of their exit criteria, but blueprints are not standard and do very little to ensure the work gets done.
- Cross-functional Prioritization aims to replace Engineering Allocations but is not prescriptive regarding how efforts involving engineers across multiple Product stages will be coordinated.

The following recording from the Rate Limit Architecture

Working Group included some organic discussion around this topic as we attempt to redefine exit criteria that will result in a successful implementation after

the working group concludes:

{{< youtube "um9deEVp618?start=1961" >}}

> Note: the video is private. Try this link
> if the embedded video is not playable.

Prior Efforts

Architecture Kickoff Working Group

The Architecture Kickoff working group was primarily concerned with defining a 3/6/12-month technical roadmap. This working group picks up by further iterating on the process for maintaining this roadmap over time and better facilitating its implementation.

Special Interest Groups

We have previously attempted to introduce a new organizational structure for facilitating cross-functional work in the form of Special Interest Groups (SIGs) (see [gitlab-com/www-gitlab-com!104378](https://gitlab.com/www-gitlab-com!104378)), but it was difficult to demonstrate incremental value with this approach.

We hope to see more success by focusing on the consolidation and streamlining of existing process and improving the success rate of working groups in their current form first.

Roles and Responsibilities

Working Group Role	Person	Title
-----	-----	-----

DRI	Marshall Cottrell	Principal, Strategy and
Operations (Technical)		
Executive Stakeholder	Christopher Lefelhocz	VP of Development
Member	Gerardo "Gerir" Lopez-Fernandez	Engineering Fellow,
Infrastructure		
Member	Joshua Lambert	Director of Product, Enablement
Member	Sam Goldstein	Director of Engineering, Ops
Member	Andrew Newdigate	Distinguished Engineer,
Infrastructure		
Member	Kamil Trzeci-ski	Distinguished Engineer, Ops and
Enablement		
Member	Philippe Lafoucrière	Security Architect, Security

Member	Grzegorz Bizon	Principal Backend Engineer, Ops
Member	Lucas Charles	Staff Backend Engineer,
Sec::Static Analysis		
Member	João Pereira	Staff Backend Engineer, Package
Member	Natalia Tepluhina	Staff Frontend Engineer, Plan
Member	Tyler Amos	Staff Backend Engineer,
Fulfillment Platform		

SECTION: company/working-groups/object-storage.md

Attributes

Property	Value
-----	-----
Date Created	November 3, 2021
Target End Date	May 31, 2022
Slack	wgobject-storage (only accessible from within the company)
Google Doc	Object Storage Working Group Meeting Agenda (only accessible from within the company)

Charter

GitLab stores three classes of user data: database records, Git repositories, and user uploaded files.

User experience, as well as contributors experience, with our file storage has room for significant improvement.

- Initial GitLab setup experience requires creation and setup of 13 buckets, instead of just 1.
- Features using file storage require contributors to think about both local storage and Object Storage which leads to friction and complexity. This often results in broken features and security issues.
- People contributing to file storage often have to write code also for Workhorse, Omnibus, and CNG.

The working group will be reducing technical debt that has been accrued over the past few years, namely removing CarrierWave and not

duplicating object storage clients in both Go and Ruby.

The working group is tasked with architecting a simplified Object Storage process and implementing the new solution.

Business goal

Improve SaaS scalability, reliability and development speed making sure object storage is available for every type of upload.

Improve feature adoption for self-managed customers, providing a single bucket configuration that works out of the box.

Object storage is a key feature in GitLab that affects engineering groups across all sections. The outcome of the working group should also make it easier for engineers to contribute to the final solution.

Scope and definitions

Object storage is a fundamental component of GitLab, providing the underlying implementation for shared, distributed, highly-available (HA) file storage.

Over time, we have built support for object storage across the application, solving specific problems in multitude of iterations. This has led to increased complexity across the board, from development (new features and bug fixes) to installation:

- New GitLab installations require the creation and configuration of several object storage buckets instead of just one, as each group of features requires its own. This has an impact on the installation experience and new feature adoption, and takes further away from boring solutions.
- The release of cloud native GitLab necessitated the removal of NFS shared storage and the development of direct upload, a feature that was expanded, milestone after milestone, to several type of uploads, but never enabled globally.
- Today GitLab supports both local storage and object storage. Local storage only works on single box installations or with a NFS, which we no longer recommend to our users and is no longer in use on GitLab.com.
- Understanding all the moving parts and the flow is extremely complicated: we have CarrierWave, Fog, Golang S3/Azure SDKs, all being used, and that complicates testing as well.
- Fog and CarrierWave are not maintained to the level of the native SDKs (e.g. AWS S3 SDK), so we end up having to maintain or monkey patch those tools to support requested customer features (e.g. <https://gitlab.com/gitlab-org/gitlab/-/issues/242245>) that would normally be "free".

- In many cases, we copy around object storage file